LOW-LEVEL DESIGN

RAG-Based Customer Support Assistant

Module-Level Implementation Reference

1. Module-Level Design

1.1 Document Processing Module

File: Cell 3 | Dependencies: langchain.document_loaders, langchain.text_splitter

```
loader = PyPDFLoader(file_name)
docs = loader.load()    # → List[Document]
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)    # → List[Document]
```

- PyPDFLoader reads each PDF page as a separate Document object with page_content and metadata
- RecursiveCharacterTextSplitter attempts splits at: \n\n → \n → ' ' → '' (recursive fallback)
- chunk_overlap=50 ensures boundary context is not lost between adjacent chunks

1.2 Embedding Module

File: Cell 4 | Dependencies: langchain_community.embeddings, sentence-transformers

```
embedding_model = HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')
# Used in three contexts:
# 1. Chroma.from_documents() — batch embed all chunks
# 2. db.similarity_search_with_score() — embed query at retrieval
# 3. semantic_grounding() — embed answer and context for cosine sim
```

- Model: sentence-transformers/all-MiniLM-L6-v2, output dim = 384
- Runs on CPU by default via HuggingFaceEmbeddings wrapper
- embed_query() is called per query (no caching implemented)

1.3 Vector Storage Module

File: Cell 4 | Dependencies: langchain_community.vectorstores.Chroma, chromadb==0.4.22

```
db = Chroma.from_documents(chunks, embedding_model)
results = db.similarity_search_with_score(query, k=5)
# returns: List[Tuple[Document, float]]
```

- Storage: In-memory only — no persist_directory configured
- Index: ChromaDB uses HNSW (Hierarchical Navigable Small World) graph internally
- Score metric: L2 (Euclidean distance); lower score = higher similarity
- k=5 candidates are retrieved before re-ranking reduces to top-3

1.4 Re-ranking Module

File: Cell 5 | Dependencies: sentence_transformers.CrossEncoder

```
reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
def rerank(query, docs, top_k=3):
    pairs = [[query, d] for d in docs]     # List[List[str, str]]
    scores = reranker.predict(pairs)        # List[float], pointwise
    ranked = sorted(zip(docs, scores), key=lambda x: x[1], reverse=True)
    return [doc for doc, _ in ranked[:top_k]]
```

- Input: (query string, list of 5 candidate passages, top_k=3)
- CrossEncoder performs joint encoding of query+passage pairs for superior relevance scoring
- Returns top-3 passages by descending score (higher = more relevant)

1.5 Retrieval Module

File: Cell 7 | Orchestrates ChromaDB + CrossEncoder

```
def retrieve(query, k=5):
    results = db.similarity_search_with_score(query, k=k)
    docs = [doc.page_content for doc, _ in results]
    scores = [score for _, score in results]
    docs = rerank(query, docs, top_k=3)   # → List[str], top-3
    return docs, scores                    # scores are pre-rerank L2 values
```

NOTE: The scores returned are the original L2 scores from ChromaDB, not the CrossEncoder re-rank scores. Confidence computation uses L2 scores.

1.6 LLM Module

File: Cells 8–10 | Dependencies: transformers, torch

```
model = AutoModelForCausalLM.from_pretrained('microsoft/phi-2',
        device_map='auto', torch_dtype=torch.float16)
tokenizer = AutoTokenizer.from_pretrained('microsoft/phi-2')

def generate_answer(prompt):
    inputs = tokenizer(prompt, return_tensors='pt',
                       truncation=True, max_length=1024).to(model.device)
    outputs = model.generate(**inputs, max_new_tokens=150,
                             temperature=0.3, top_p=0.9, do_sample=True)
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return response[len(prompt):].strip()   # strip echo of prompt
```

- device_map='auto': Places model on GPU if available, else CPU
- float16: Reduces GPU memory footprint by ~50% vs float32
- Temperature=0.3: Near-deterministic, reduces hallucination risk
- top_p=0.9: Nucleus sampling for slight diversity
- max_new_tokens=150: Caps response length; prevents runaway generation
- Prompt is stripped from the output (response[len(prompt):]): Phi-2 echoes the prompt

1.7 Query Processing Module

File: Cells 9, 11, 12 | Helper functions

```
def rewrite_query(query):
    prompt = f'Rewrite this into a clear, specific question for document
search.\nQuery: {query}\nRewritten:'
    return generate_answer(prompt)

def detect_intent(query):
    q = query.lower()
    if any(x in q for x in ['refund','complaint','issue','problem','not
working']): return 'complaint'
    elif any(x in q for x in ['summarize','summary','overview']): return
'summary'
    elif any(x in q for x in ['how','steps','process','procedure']): return
'process'
    elif any(x in q for x in ['what','when','where','which']): return 'faq'
    return 'unknown'

def compute_confidence(scores):
    return sum(scores) / len(scores) if scores else 0.0
```

IMPORTANT: detect_intent() is applied to the REWRITTEN query (after rewrite_query), not the original user query. rewrite_query() is wrapped in try/except — if Phi-2 fails, original query is used.

1.8 Grounding Verification Module

File: Cells 6, 12

```
def semantic_grounding(answer, context):
    emb1 = embedding_model.embed_query(answer)
    emb2 = embedding_model.embed_query(context[:1000])
    similarity = dot(emb1, emb2) / (norm(emb1) * norm(emb2))
    return similarity > 0.55

def is_answer_grounded(answer, context):   # Defined but NOT USED in chat()
    answer_words = set(answer.lower().split())
    context_words = set(context.lower().split())
```

```
        overlap = len(answer_words.intersection(context_words))
        return overlap / (len(answer_words) + 1) > 0.3
```

NOTE: Two grounding functions are defined. chat() uses semantic_grounding() (cosine similarity, threshold 0.55). is_answer_grounded() (word overlap, threshold 0.3) is defined but never called — dead code.

2. Data Structures

2.1 Document Object (LangChain Document)

Field	Type	Description
page_content	str	Text content of the page/chunk
metadata	dict	e.g. {'source': 'file.pdf', 'page': 0}

2.2 Chunk Schema

Field	Type	Value / Constraint
page_content	str	Max 500 chars per chunk; min ~1 char
metadata	dict	Inherited from source Document (page number, source path)
overlap	int	50 characters shared with adjacent chunk

2.3 Embedding Vector

Property	Value
Type	List[float] (Python) / numpy array
Dimension	384 (all-MiniLM-L6-v2 output size)
Normalization	L2-normalized by sentence-transformers
Storage	ChromaDB manages internally (not exposed as Python object)

2.4 Retrieval Result

Field	Type	Description
docs	List[str]	Top-3 passage strings after re-ranking
scores	List[float]	Top-5 L2 scores from ChromaDB (pre-re-rank)
context	str	' '.join(docs) — concatenated retrieval context

2.5 chat() Function State (Implicit)

There is no formal LangGraph State object. The chat() function uses local Python variables as implicit state:

Variable	Type	Role
query	str	Rewritten query (mutated from original via try/except)

Variable	Type	Role
docs	List[str]	Top-3 retrieved passages after re-ranking
scores	List[float]	L2 scores for confidence computation
context	str	Joined passages as LLM context
intent	str	Detected intent: complaint/summary/process/faq/unknown
confidence	float	Mean L2 score from retrieval
answer	str	Raw LLM generation output
grounded	bool	Semantic grounding check result (cosine sim > 0.55)
response	str	Final assembled response string

3. Workflow Design

3.1 LangGraph Clarification

langgraph==0.0.32 is installed as a dependency but no StateGraph, Node, or Edge objects are instantiated in the notebook. The workflow is entirely implemented via the imperative chat() function. The workflow description below maps chat() steps to equivalent LangGraph node concepts for design documentation purposes.

3.2 Equivalent Node Mapping

Logical Node	Implementation	Responsibility
query_rewrite_node	rewrite_query()	Refines user query for better retrieval using Phi-2
retrieve_node	retrieve(query, k=5)	Fetches top-5 chunks, re-ranks to top-3
intent_node	detect_intent()	Classifies query intent via keyword matching
confidence_node	compute_confidence(scores)	Computes mean L2 score as confidence proxy
context_check_node	len(context.strip()) < 50	Guards against empty retrieval
generate_node	build_prompt() + generate_answer()	LLM answer generation from context
grounding_node	semantic_grounding()	Validates answer is semantically grounded
router_node	if/elif chain in chat()	Routes to answer, fallback, or escalation
output_node	response assembly	Appends DEBUG INFO and SOURCES to answer
hitl_node	[ESCALATED] string tag	Signals human intervention required

3.3 State Transition Flow

```
START
  |
  ▼
query_rewrite_node  ──[try/except]──▶ (original query if fail)
  |
  ▼
retrieve_node  ──▶ docs[], scores[]
  |
  ├─▶ intent_node  ──▶ intent string
  └─▶ confidence_node  ──▶ float
  |
  ▼
context_check_node
  ├─[EMPTY]──▶ hitl_node  ──▶ END
```

```
└─[OK]──▶ generate_node
            │
            ▼
          grounding_node
            │
            ▼
          router_node
            ├─[complaint]──────────▶ hitl_node  ──▶ END
            ├─[not grounded]───────▶ fallback_node ──▶ END
            ├─[confidence < 0.4]──▶ hitl_node  ──▶ END
            ├─[unknown intent]────▶ clarify_node ──▶ END
            └─[all pass]───────────▶ output_node ──▶ END
```

4. Conditional Routing Logic

Priority	Condition	Check Type	Output
1 (highest)	not docs OR len(context.strip()) < 50	Hard guard	[ESCALATED] No relevant information found in document
2	intent == 'complaint'	Intent match	[ESCALATED] Complaint detected → Human support required
3	not grounded (cosine sim ≤ 0.55)	Semantic check	Soft fallback: rephrase request (no ESCALATED tag)
4	confidence < 0.4	Threshold check	[ESCALATED] Low confidence retrieval
5	intent == 'unknown'	Intent fallback	Clarification prompt to user (no ESCALATED tag)
6 (default)	None of the above	Pass-through	Answer + DEBUG INFO + SOURCES

IMPORTANT NOTE on Routing Order: The complaint check (priority 2) runs AFTER the LLM has already generated an answer (generate_answer() in step 4). This means Phi-2 is invoked even for complaint queries, wasting compute. A production system would place intent check before LLM generation.

5. HITL Design

5.1 Trigger Conditions

Trigger	Condition Code	Escalation Message
Empty retrieval	not docs or len(context.strip()) < 50	'[ESCALATED] No relevant information found in document'
Complaint intent	intent == 'complaint'	'[ESCALATED] Complaint detected → Human support required'
Low retrieval confidence	confidence < 0.4	'[ESCALATED] Low confidence retrieval'

5.2 Non-Escalation Fallbacks

Condition	Response Type	Message
Semantic grounding fails	Soft fallback (no HITL tag)	Rephrase prompt returned to user
Unknown intent	Clarification request	Ask user to rephrase based on document

5.3 HITL Implementation Gaps

- No actual handoff to human agent queue (no webhook, email, or ticket system)
- No logging of escalated queries for agent review
- No mechanism to inject human agent's response back into the system
- No session ID or user identification attached to escalation
- The [ESCALATED] prefix is a string convention — relies on caller to parse it

6. Interface Design

6.1 Input Schema

Parameter	Type	Constraints	Source
query	str	Non-empty string	CLI input() in while loop
'exit' or 'quit'	str	Terminates the while loop	CLI sentinel values

6.2 Output Schema

Response Type	Format	Example
Normal answer	answer + \n\n--- \n[DEBUG INFO]\n... + \n[SOURCES]\n...	Full answer with metadata
HITL escalation	'[ESCALATED] <reason>'	'[ESCALATED] Complaint detected → Human support required'
Soft fallback	Plain string message	'I'm not confident this answer is supported...'
Clarification	Plain string message	'Can you clarify your question based on the document?'

7. Error Handling

Failure Scenario	Handling in Code	Outcome
rewrite_query() throws exception	try/except passes, original query used	Graceful degradation, no crash
No relevant chunks retrieved	Explicit check: not docs or context < 50 chars	Returns [ESCALATED] message
Semantic grounding fails	Soft fallback string returned	User prompted to rephrase
Low confidence (< 0.4)	[ESCALATED] message returned	Human escalation triggered

Failure Scenario	Handling in Code	Outcome
Embedding failure	Not handled — would raise unhandled exception	System crash
Phi-2 generation failure (answer)	Not handled — would raise unhandled exception	System crash
ChromaDB initialization failure	Not handled — script would fail at Cell 4	Colab cell error